Applied Intelligence (2023) 53:405–422
https://doi.org/10.1007/s10489-022-03456-w

Deep reinforcement learning for the dynamic and uncertain vehicle routing problem

Weixu Pan[1] · **Shi Qiang Liu**[1]

Accepted: 1 March 2022 / Published online: 18 April 2022
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2022

Abstract
Accurate and real-time tracking for real-world urban logistics has become a popular research topic in the field of intelligent transportation. While the routing of urban logistic service is usually accomplished via complex mathematical and analytical methods. However, the nature and scope of real-world urban logistics are highly dynamic, and the existing optimization technique cannot precisely formulate the dynamic characteristics of the route. To ensure customers' demands are met, planners need to respond to these changes quickly (sometimes instantaneously). This paper proposes the formulation of a novel deep reinforcement learning framework to solve a dynamic and uncertain vehicle routing problem (DU-VRP), whose objective is to meet the uncertain servicing needs of customers in a dynamic environment. Considering uncertain information about the demands of customers in this problem, the partial observation Markov decision process is designed to frequently observe the changes in customers' demands in a real-time decision support system that consists of a deep neural network with a dynamic attention mechanism. Besides, a cutting-edge reinforcement learning algorithm is presented to control the value function of the DU-VRP for better training the routing process's dynamics and uncertainty. Computational experiments are conducted considering different data sources to obtain satisfactory solutions of the DU-VRP.

Keywords Graph neural network · Reinforcement learning · Partial observation Markov decision process · Vehicle routing problems

1 Introduction

E-commerce giant enterprises such as Alibaba and JD have recently invested a large number of resources in group-on buying, especially for fresh food. This trend is becoming more popular and receiving more attention as the next-generation research topic [1]. Meanwhile, to incorporate the dynamic dispatching policy and the real-time routing strategy for fresh food delivery services, vehicle routing is a cornerstone activity that is frequently related to the decision-making processes of several real-world transportation systems [2]. In particular, meeting customers' expectations and optimizing delivery requirements are expected to ensure food businesses a competitive advantage because food delivery accounts for a large portion of operational costs [3]. Although many researchers have been working on deterministic or static vehicle routing problems (VRPs), little attention has been paid to dealing with uncertainty and dynamics in the VRPs simultaneously [4–6]. Furthermore, in classical VRP models, the delivery requirements in VRPs are assumed as static, implying that some essential real-world constraints (i.e., dynamic and uncertain demands of customers) are neglected. This assumption may result in the inapplicability of real-world implementation. For example, in the food delivery industry, the failure of meeting the uncertain demands punctually would result in losing the freshness of food.

The basic VRP was initially formulated by Dantzig and Ramser as a generalization of the Traveling Salesman Problem (TSP) [7]. The VRP has been known as among the most difficult problems in combinatorial optimization [8]. Due to its significance in practice, several VRP variants have been widely studied since the 1950s till now [9]. In the following, two extended VRPs are described by indicating

Weixu Pan and Shi Qiang Liu contributed equally to this work.

✉ Shi Qiang Liu
samsqliu@fzu.edu.cn

Weixu Pan
n190710007@fzu.edu.cn

[1] School of Economics and Management, Fuzhou University, Fuzhou, 350108, Fujian, China

their main differences from the basic VRP problem. One is the Dynamic VRP (DVRP), which differs from the basic VRP by supposing that some values associated with customers, locations, and vehicles may change dynamically [10]. Such a proposition is common in today's vibrant production and transportation environment. Another is the Dynamic and Uncertain VRP (DU-VRP), which is an extension of the DVRP by incorporating both uncertainties and dynamics in the vehicle routing procedure simultaneously. In practice, it is difficult to know the exact and fully-confirmed information in advance. For example, dispatchers have to update real-time information regarding the vehicles' positions and consider the uncertainty of each customer's demand. In this study, we focus on the DU-VRP as it has a wider range of applicability.

More preciously, we consider to solve the DU-VRP by a Deep Reinforcement Learning (DRL) approach. A small-size DU-VRP example shown in Fig. 1 reveals how demand uncertainty affects the routes of vehicles in a consistent fashion and indicates the importance of real-time communication under a DU-VRP environment. Real-time communication needs to collect uncertain information and assign tasks in a dynamic way. In the DU-VRP, new customers arrive dynamically in the service areas. Figure 1 is drawn to illustrate the route execution of a single vehicle in an uncertain and dynamic environment. An initial route intends to visit the currently-known customers (i.e., Nodes A, B, C, D, E) before the vehicle leaves the depot at the time t_0. While the vehicle executes this route, some requests from customers (i.e., Nodes L, X, Y, Z, K) may appear at times t_1, t_2, t_3 in order. Each customer has a given demand by themselves. Furthermore, consumer demands may be uncertain and dynamically changing, and the requests that result from their demands are highly uncertain. In this case, this initial route may be adjusted to meet changes of the environment, e.g., the executed route at the time t_f may be changed to be A $\rightarrow$ L$\rightarrow$ B $\rightarrow$ X $\rightarrow$ C $\rightarrow$ Y $\rightarrow$ D $\rightarrow$ Z $\rightarrow$ E $\rightarrow$ K.

A real-time communication scheme under a DU-VRP environment is depicted in Fig. 2. The dispatcher can tailor the routing to the dynamic environment at each time step. In a dynamic DU-VRP environment, a dispatcher is treated as an agent to give instructions to the vehicles. The dispatcher makes a sequence decision to fulfill the needs of each customer when the vehicle is ready (e.g., Customer A highlighted by the first double-headed arrow in Fig. 2). When the location of the customer results in environmental changes, the DU-VRP system receives environmental information in real time and makes a potential decision on updating a new route.

An essential challenge in solving the DU-VRP is enabling the dispatchers to learn from the customer-vehicle data in real time. Furthermore, consumer demands may be uncertain and dynamically changing, and the requests that result from their demands are highly uncertain. Each customer-vehicle-based time is associated with the high-dimensional data to address the dynamic constraints. These realistic constraints may be designed by the Markov decision process (MDP), in which the dispatcher can select an action in a dynamic environment [11]. However, conventional optimization methods are inapplicable to handle large-scale and complex MDPs, especially for collecting or controlling the components of the so-called value function in MDP, due to the so-called "curse of data dimensionality" that happens when the accuracy of the model decreases as the number of features increases. On the other hand, the dispatcher determines the daily route of vehicles in a given map while the locations of potential customers may differ. One primary concern in this practical setting is that uncertainty data need to be collected repeatedly with different customers' demands in the map sequentially. Traditional optimization methods (e.g., robust optimization approach, distributionally robust optimization approach and chance-constrained programming approach) may not be effective in dealing with system dynamics for real-time decision-making [12]. Hence, to reduce the DU-VRP data dimensionally, the development of a cutting-edge DRL approach through investigating a learning-while-optimizing strategy is needed.

The recent progress in machine learning (ML) shows that deep reinforcement learning (DRL), which integrates deep learning (DL) and reinforcement learning (RL) techniques, is able to handle such a kind of routing problems with

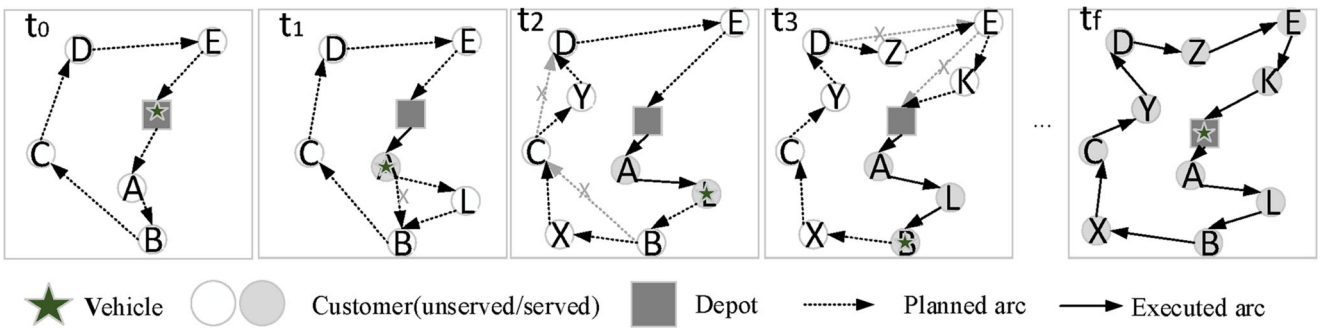


Fig. 1 Illustration of the DU-VRP

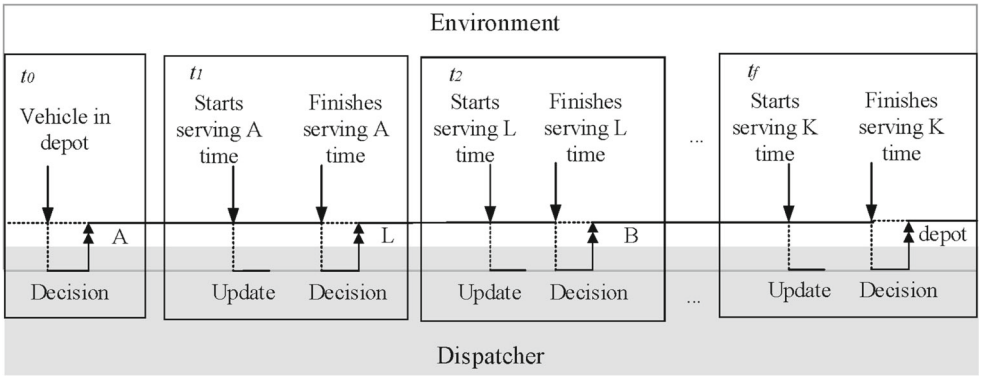


Fig. 2 Timeline of events for the DU-VRP

dynamics and uncertainties competently [13]. While reinforcement learning is a model-free approach in which an agent learns the best rules from its environment [14]. The policy of the MDP is fully dependent on the current state at each time step. Partially observable Markov decision processes (POMDPs) are the improved MDPs in which the state at each time step is not known with inerrability, that is, the state is not completely observable [15, 16]. Graph-based POMDPs are an extension of standard POMDPs that add routing and graph information to the state space, action space, and reward structure.

Graph-based POMDPs are an appropriate solution for the DU-VRP, because a dispatcher does not fully receive customer information across the service area but just at certain points. As a result, graph-based POMDPs make it easier to connect dynamic routing challenges to the graph-based approaches that are commonly used to address them – design and amend routes as new information is learnt. The end-to-end architecture has been proposed to address the underlying dynamics and uncertainties in real-world transportation systems, owing to its advantages in storing previous data and producing multi-step predictions [17, 18]. To adapt partially observable decision-making for the DU-VRP situations, we tempt to model this problem as a group-based POMDP and develop a novel DRL method to solve the DU-VRP efficiently.

The main contributions of this study are highlighted as follows.

- This study is among the first to investigate how to develop a novel end-to-end DRL framework for the DU-VRP.
- We design a novel graph-based POMDP model to reflect the dynamic environment and uncertain customers' demands in the DU-VRP.
- An adaptive policy is designed to train the objective function value and enhance the learning performance, especially for the graph-based POMDP model.
- The proposed DRL methodology is validated by baseline comparisons and sensitivity analysis using real-world data.

The remainder of this paper is organized as follows. We systematically integrate the recent applications of DRL and RL methods on the VRPs and related transportation cases in Section 2. Section 3 defines and formulates the DU-VRP by a graph-based POMDP and proposes a novel DRL methodology for the graph-based POMDP. Section 4 conducts computational experiments to validate the proposed DRL methodology's performance. The conclusions of this study and future research directions are provided in the last section.

2 Literature review

While software and hardware computing capabilities have advanced in recent years, academia and business have increasingly focused on using machine learning approaches to address combinatorial optimization issues [19]. To analyze large-scale data, DL's perceptual skills and RL's reasoning cognition may be used to combinatorial optimization issues. It will outperform standard approaches in terms of resilience and generality. The two key objectives for using deep reinforcement learning are to approximate and explore innovative policies [17].

Traditional optimization methods may be utilized to deal with the uncertainties in transportation systems and provide optimum VRP solutions. On the other hand, these technologies may not be effective in dealing with system dynamics for real-time decision-making. Because these methods need longer to create solutions, they cannot respond to system dynamics as quickly as reinforcement learning approaches, particularly when dealing with routing problems [20]. As a result, RL may be a superior option to classic optimisation approaches since it can be taught with real-world data to interact more effectively with system dynamics and consumer demand uncertainties, resulting in quicker real-time routing and dispatching choices.

Reinforcement learning is a popular technique for dealing with dynamic decision-making difficulties in the face of uncertainty. This portion of the review focuses mostly on

model-free RL algorithms, which may be divided into three categories. There are three types of RL algorithms: 1) value-based algorithms (e.g., Q-learning, Deep Q-learning (DQN)) [21, 22], 2) policy-based algorithms (e.g., REINFORCE) [23], and 3) actor-critic algorithms, which integrate value-based with policy-based algorithms [24]. Several new techniques for solving routing problems have been proposed in the literature, including: 1) recurrent neural network (RNN) model with an attentional mechanism based on an encoder-decoder architecture [25–27], 2) graph embedding network models to express the policy that captures a node's property in the context of its graph neighbourhood [23, 28, 29], and 3) the representation of the environment based on the MDP [11, 30, 31].

Deep Reinforcement learning enhances classic RL by employing deep neural networks (DNNs) as function approximators, which has shown to be successful in a variety of control issues. Asynchronous techniques are given as an alternative, and they can execute concurrent and independent computing tasks. Minh et al. [32] introduced the asynchronous advantage actor-critic (A3C), which allows many actor-critic agents to train in parallel. Each agent has its own set of parameters and interacts with the environment in a unique way. A global network receives updates from each agent and combines them asynchronously to create a global policy. Advantage actor-critic(A2C) is a synchronous and deterministic variation of A3C. Experiments by Minh et al. [32] suggested that A3C can train quicker than DQN.

We systematically integrate the recent applications of DRL and RL methods on the VRPs and related transportation cases, including urban freight deliveries and on-demand ridesharing for passenger transportation. On-demand ridesharing concerns based on DRL are investigated from two perspectives: 1) order dispatching that matches rider demands with available cars; and 2) vehicle repositioning is a proactive method of balancing vehicle supply and demand by moving idle automobiles from one zone to another. Existing DRL papers for urban freight delivery, order dispatching, and vehicle repositioning are reviewed in the next three paragraphs. We shall synthesize the problem characteristics and DRL approaches, highlighting the novelty of our paper.

The travelling salesman problem (TSP), which involves the routing of a single vehicle, is a rudimentary variant of VRP problems. Archetti et al. [33] considered the dynamic TSP with stochastic release dates as an MDP and proposed an RL-based approach for solving this TSP. Only spatial information of nodes is relevant in the TSP difficulties discussed above. DRL actions are used to gradually develop the vehicle path by adding nodes one at a time. VRPs have been created using cutting-edge approaches based on approximate algorithms, but they still have issues with long running time and poor solution quality when dealing with huge problems. The goal is to determine the best vehicle delivery routes and technician service routes in order to reduce the overall distance traveled by the delivery and service routes. Qiu et al. [34] proposed a DRL method to solve it. Traditional approaches face a contradiction between computational efficiency and solution quality in the face of the present rapid rise in logistical needs. The majority of current optimization methods are taking a long time to compute in urban transportation networks. To address these issues, Zhao et al. [27] developed a new DRL model that includes an encoder-decoder network with an attentional mechanism. To efficiently tackle the problem, Zhang et al. [25] designed the Multi-Agent Attention Model, a unique RL method that can solve routing problems instantaneously while benefiting from extensive offline training. Yu et al. [35] presented a novel DRL-based technique for developing routes in the shortest time possible. Their method combines the heuristic and the DNN to train the vehicle routing data. In the framework of crowdsourced urban delivery, Ahamed et al. [36] designed a new DRL-based technique for solving the difficulty of distributing shipment orders to ad hoc couriers. Silva et al. [37] presented a multi-agent RL approach to solve the VRP by using the RL ideas, especially the Q-Learning algorithm to improve the agent's self-adaptive skills and to adapt quickly to the unique characteristics of the situation to be handled by the architecture. The goal of this framework is for each agent to be able to change their activities based on their interactions with other agents and the environment. Kang et al. [38] considered a graph-based DRL algorithm that handles the acceptance of order requests and determines the route itineraries of multiple vehicles.

In view of the important challenges to the efficient dispatching of orders, Zou et al. [39] designed a DRL technique based on DQN for solving the online food ordering delivery problem. Kullman et al. [30] addressed the situation of a ride-hailing service operator managing a fleet of electric vehicles. To maximize the profit, the operator must allocate cars to demands as they come, as well as recharge and reposition vehicles in advance of future requests. Kullman et al. [30] applied DRL to tackle this challenge, creating policies that make decisions based on Q-value approximations learnt by deep neural networks. On a ride-sharing platform, ride dispatching is a core operation task that matches drivers to trip-requesting clients on a constant basis. Wang et al. [40] offered an MDP model for the ride dispatching problem and presented learning techniques for improving the existing policy for drivers based on DQN with action search. Liang et al. [41] provided a scalable framework for promoting real-time taxi operations that incorporates RL with a centralized programming structure. To construct a near-optimal strategy, Turan et al. [42] developed the DRL method via Proximal Policy Optimization (PPO). Chen et

al. [43] used fleets of drones and vehicles to solve the challenge of dynamic same-day delivery. They describe a DQN strategy that takes advantage of both state and action space data. Liu et al. [44] enhanced DRL for food route delivery by leveraging Dijkstra's algorithm to determine the existing policy and compute the vehicle tour in each iteration, allowing the system to learn from past trajectories more effectively.

In view of the important challenges to the efficient dispatch of vehicle repositioning orders, Liu et al. [45] designed a novel RL method to handle the path of driving vehicles in emergency situations. Tang et al. [46] offered an RL approach to solve an automated electric taxi fleet's online routing problem with the consideration of the dynamic demands of the taxis. Koh et al. [47] designed a novel DRL approach to solve a real-time vehicle navigation problem. Mao et al. [48] designed a novel DRL technique to solve a real-world taxi dispatching service problem. They specifically include the RL framework, which focuses on how to find an optimum policy when the MDP technique is unavailable. As a result, rather than depending on any previous model knowledge, the agent in RL will communicate directly with the transport system and change the control approach. AI-Abbasi et al. [49] introduced a distributed model-free pool (DeepPool) that learns optimal allocation strategies by observing the environment using DQN approaches. DeepPool also incorporates travel demand information with DL models to control vehicle dispatching for enhanced ride-sharing services. Basso et al. [50] presented a Safe RL approach to solve the Dynamic Stochastic Electric Vehicle Routing Problem. The main concept is to use Monte Carlo simulations to learn offline about stochastic consumer demands and energy usage in order to design the route predictably and securely online. Guo et al. [51] focused on establishing a real-time routing system to optimize the rentability of car service providers, given the complicated metropolitan situation and unpredictable social occurrences (e.g., taxi drivers). To deal with this challenge, an RL framework with a dynamic attention mechanism and a DNN is presented by Guo et al. [51]. Because of the variety of settings, the ambiguity in sensor data, and the complicated interaction with other road users, operational decisions for vehicles are difficult. In the form of DRL with Monte Carlo tree search, Hoel et al. [52] offered a comprehensive framework for tactical decision making that integrates the notions of planning and learning.

We compare our current scenario with a wide variety of existing methods. Relevant papers solving VRPs under various scenarios are summarized in Table 1. Below is a summary of the existing methods from each category. The third column shows the observable characteristics of the state. It can be observed that most of the existing methods are for perfect state observability. The fourth column shows the characteristics of online or offline. With a few exceptions, as indicated in Table 1, the bulk of research in both urban freight delivery and order dispatching operates in a full-state observability environment and online fashion.

These existing works did not optimize over time and in the face of uncertainty by capturing the dynamic aspects of customers' demands. Because the underlying mathematical model is a non-trivial sequential stochastic optimization model that is impossible to solve, the bulk of the studies is based on full observability techniques without a mathematical model for the suggested dispatch policies. Some studies provided a mathematical model with an assumption of operating in a full state observability environment but ignoring the uncertainty in the future customer's demand. Few works [15, 52] applied the RL approach to optimise different forms of routing for partial observability. Moreover, each of these studies focused on a certain set of decisions that must be made in order to operate a fleet of vehicles. Based on the above literature review, we come to some insightful conclusions on the current research gaps and then highlight the main innovative aspects of our study.

- Till now, academic papers on the application of DRL (RL) to handle the dynamic and uncertain issues in the VRP system have rarely been found. To the best of our knowledge, this study is pioneering to explore how to develop a novel DRL methodology for the DU-VRP.
- In the literature, most of DRL (DL) methods are based on full state observability. Instead, we adopt a new graph-based partially observable Markov decision process (POMDP) to formulate the partial state observability of the uncertainty and dynamics in the DU-VRP system.
- Most of the existing research applies single neural networks (e.g., NN, RNN, etc.) to approximate the value function in their end-to-end DRL framework. In comparison, we propose a cutting-edge graph-embedding network to tackle the dynamic customers' coordinates in the DU-VRP environment.

3 DRL methodology for DU-VRP

This section explains how the DRL methodology is developed for the DU-VRP. First, the DU-VPR is modelled and described by the POMDP in Section 3.1. Then, we propose a cutting-edge DRL methodology to solve the DU-VRP, combining a DL approach with a graph embedding network for the DU-VRP and an RL approach to optimize the graph embedding network in Section 3.2.

Table 1 Summary of DRL applications to VRPs

Problem type	Authors	State observability	Online or Offline	Method
Urban freight delivery	Qiu et al. [34]	Full	Offline	Actor-critic
	Zhao et al. [27]	Full	Offline	Actor-critic
	Zhang et al. [25]	Full	Offline	Multi-agent DRL with attention
	Ahamed et al. [36]	Full	Offline	DQN
	Silva et al. [37]	Full	Offline	Q-Learning
	Yu et al. [35]	Full	Online	REINFORCE
	Kang et al. [38]	Full	Offline	DQN
Order dispatching	Wang et al. [40]	Full	Online	DQN
	Liang et al. [41]	Full	Online	DQN and A2C
	Kullman et al. [30]	Full	Online	DQN, DDQN, D3QN
	Zou et al. [39]	Full	Online	Double DQN
	Turan et al. [42]	Full	Online	PPO
	Chen et al. [43]	Full	Online	DQN
	Liu et al. [44]	Full	Offline	Double DQN
Vehicle repositioning	Liu et al. [45]	Full	Offline	Q-Learning
	Al-Abbasi et al. [49]	Full	Online	DQN
	Tang et al. [46]	Full	Online	Approximate Dynamic Programming
	Koh et al. [47]	Full	Online	Enhanced DQN
	Basso et al.[50]	Full	Online	Q-learning
	Mao et al. [48]	Full	Offline	Actor-critic
	Guo et al. [51]	Full	Online	Temporal difference learning
	Hoel et al. [52]	Partial	Offline	Monte Carlo tree search

The third column shows the observable characteristics of the state. It can be observed that most of the existing methods are for full state observability

3.1 Modelling the DU-VRP by POMDP

In contrast to the static VRP, real-world applications are often produced in a dynamic environment. Typically, in the dynamic VRP, the information available to the dispatchers may change during the execution of the routes with new demands. Therefore, DU-VRP situations do not necessarily have the Markov property because of the temporal dependence of customers' demands. The problem with such an implementation is responding to new demands while optimizing the total traveling cost. Due to the difficulty of uncertainties and dynamics, static mixed integer programming (MIP) models are not directly applicable for modeling the DU-VRP. Graph-based POMDPs are an extension of standard POMDPs that add routing and graph information to the state space, action space, and reward structure. As a contribution to the literature, we present a novel graph-based partially observed Markov decision process (POMDP) to model the DU-VRP. Figure 3 illustrates the sequential process of the POMDP for modeling the DU-VRP by the notations defined as follows.

- Time steps

  Vehicle routings are considered to be optimized over a finite time horizon for each time step $t = 0, 1, 2, \ldots, T$, where T represents the length of a morning or an evening peak period.
- States

  The graph-based POMDP is associated with a directed graph model, $G = (\mathbb{C}, E)$, where $\mathbb{C} = \{c_0, c_1, \ldots, c_n, c_{n+1}\}$ represents a set of nodes; c_0 and c_{n+1} are the source and sink depots; c_i is customer $i (i = 1, \ldots, n)$. It is assumed that each customer is associated with only one node. The element $E = \{(i, j), c_i, c_j \in \mathbb{C}\}$ represents a set of arcs. A fleet of non-homogeneous vehicles $k = 1, \ldots, m$ is regarded as a set of agents. Each agent (vehicle) starts its route at the dummy source depot c_0 and stops at the dummy sink depot c_{n+1}. Each customer i has a demand d_i. A new binary variable u_{ik} that equals 1 if an agent k meets the demand of customer i , and 0 otherwise, is defined. A new binary variable x_{ijk} that equals 1 if an agent k travels from customer i to customer j; 0, otherwise, is defined.

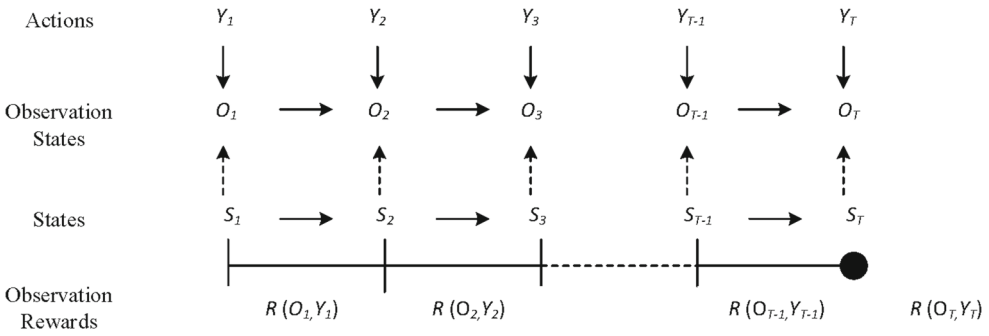


Fig. 3 Illustration the sequential process of the POMDP for DU-VRP

The graph is transformed into the set of states $\mathcal{S} = \{S_t, t = 1, \ldots, T\}$ at time steps. Each state S_t consists of three elements: a set of customers in the current state $\mathbb{C}_t$; vehicles' location $\mathcal{M}_t$; and customers' demands $\mathcal{D}_t$ at time step t. Note that $\mathbb{C}_t$ is composed of $\mathbb{C}_t^{next}$ and $\mathbb{C}_t^{new}$, $\mathbb{C}_t \in \mathbb{C}$, where $\mathbb{C}_t^{next}$ represents the unserved customers at time step t, and $\mathbb{C}_t^{new}$ represents the new customers at time step t. Note that the vehicles' locations are represented by $\mathcal{M}_t = \{m_k^t, k = 1, \ldots, n\}$, where m_k^t defines the location of vehicle k at time step t. Note that the customers' demand $\mathcal{D}_t$ is obtained dynamically during the servicing of the new customers. With the above formulations, the state space of the DU-VRP is denoted as $S_t = \{(\mathbb{C}_t, \mathcal{M}_t, \mathcal{D}_t) \mid \mathbb{C}_t \in \mathbb{R}_+^n, \mathcal{M}_t \in \mathbb{R}_+^n, \mathcal{D}_t \in \mathbb{R}_+^n$.

- Observation states

  The DU-VRP decisions are dynamically executed in a real map, in which the demand of customers is inaccessible for the agents indeed. As a result, the agent has to infer a probability distribution over the set of possible states based on a set of observations. The observation states are associated with an observed current customers' location $\tilde{\mathcal{M}}_t$ and observed customers' demands $\tilde{\mathcal{D}}_t$ at time step t,defined as $O_t = \{(\tilde{\mathcal{M}}_t, \tilde{\mathcal{D}}_t) \mid \tilde{\mathcal{M}}_t = \{\tilde{m}_{ik}^t, i, k = 1, \ldots, n\}, \tilde{\mathcal{D}}_t = \{\tilde{d}_i^t, i = 1, \ldots, n\}$ where $\tilde{m}_{ik}^t$ is the observed distance between new customer i and vehicle k at time step t, and $\tilde{d}_i^t$ is the observed demand of customer i at time step t. Thus, the whole observation space is defined as $\mathcal{O} = \{O_t, t = 1, \ldots, T\}$.
- Action

  Notation Y_{t+1} is defined as a pointer referring to the action of the available customers at time step t. The action sequence is denoted as $\mathcal{Y} = \{Y_1, \ldots, Y_t\}$ recording the past information of actions.
- Transition probability

  The transition probability $\mathbb{T}(s' \mid s, y) = \Pr(S_{t+1} = s' \mid S_t = s, y_t = y)$ represents the probability of a given action y_t from the current state s to the next state s' at time step t, for all $s', s \in \mathcal{S}$, and $y \in \mathcal{Y}$.
- Observation probability

  The observation transition probability $p(o' \mid s, y) = \Pr(O_{t+1} = o' \mid S_t = s, Y_t = y)$ represents the probability of a given action Y_t from the current state S_t to the observation state O_{t+1} at time step t, for all $o' \in \mathcal{O}, s \in \mathcal{S}$, and $y \in \mathcal{Y}$.
- Observation Reward function

  The reward function is defined as $R(O_{t+1}, Y_{t+1}) = V(O_{t+1}, Y_{t+1}) - P(O_{t+1}, Y_{t+1})$, where $V(O_{t+1}, Y_{t+1})$, $P(O_{t+1}, Y_{t+1})$ are the complete observation reward and partial observation penalty functions, respectively. $V(O_{t+1}, Y_{t+1}) = \sum_{k=1}^{m} \sum_{i=1}^{n} u_{ik} - \sum_{k=1}^{m} \sum_{i=1}^{n} \sum_{j=1}^{n} c_{ij} x_{ijk}$,

  is calculated as the reward of demand fulfillment and the negative value of the distance between two customers during the action. $P(O_{t+1}, Y_{t+1}) = C_1 \sum_{k=1}^{m} \max\{\sum_{i=1}^{n} d_i \sum_{j=1}^{n} x_{ijk} - q_k, 0\}$, where the q_k and d_i represent the capacity of vehicle k and demand of customer i, respectively. In this penalty function, the penalty term with coefficient C_1 guarantees capacity constraints of each vehicle. The immediate reward R_t is generated after the vehicle moved from O_t to O_{t+1} taking action y_{t+1}.
- Policy

  The policy is a stochastic rule π, which controls the tour to serve all customer requests at each time step.
- State Value function

  The state value function is defined as $V_\pi(o) = \mathbb{E}_\pi[\sum_{k=0}^{\infty} R_{t+k+1} | O_t = o]$, for all $o \in \mathcal{O}$, which is the expected return when starting in state o executing a policy π.
- Action Value function

  The action value function $\mathbb{Q}(o, y) = \mathbb{E}[R_{t+1} + v(O_{t+1}) \mid O_t = o, Y_t = y]$, represents the expected return for taking action y in state o, for all $o \in \mathcal{O}$, $v \in V, y \in \mathcal{Y}$.

3.2 DRL methodology

This subsection proposes a cutting-edge graph embedding network model (GENM), which is fundamentally an encoder-decoder structure. The modelling specifics of the proposed DRL methodology for the DU-VRP are described as follows.

3.2.1 Overview of DRL methodology

The main structure of the proposed DRL methodology is illustrated in Fig. 4. At the beginning of each time slot, the dispatchers receive data from the information dispatch center (IDC) that collects the current customer demand in real time. The main function of the IDC is to update the overall flow of the information about the locations and demands according to changes in the dynamic environment. The environment information can add a set of observations to the POMDP model through a DL approach, which is designed to obtain the observation values. Then, the agent observes the environment at each state and produces an action through an encoder-decoder network. Finally, the RL approach is performed to implement the optimized tour strategy.

According to the above analysis, the proposed DRL methodology for the DU-VRP is based on the bidirectional interfaces with a combination of DL and RL approaches. The DL approach develops tour strategy through the graph from the tour graph creation when receiving the real-time information of customers' demands, and then delivers the tour strategy to the RL approach. Subsequently, the RL approach optimizes the tour strategy for each vehicle in such a way that each vehicle implements its tour strategy to drive towards the next node. Besides, the DL approach transmits a request for the current state to the POMDP model with the updated data.

3.2.2 A DL approach

We design a DL approach containing the graph embedding network, RNN, attention mechanism with pooling layer. The DL approach is introduced to construct a real-time controller which can estimate the state and action values using the encoder-decoder network. In the input layer of the encoder-decoder network, we design a graph embedding method to analyze the characteristics of the DU-VRP at each state in Fig. 5. The Rectified Linear Unit (ReLU) activation function is the linear unit, as it can accelerate the convergence of the embedding node. This graph-based method has been applied to describe the relationships between each pair of nodes in the graph by compressing an adjacency matrix with the consideration of customer coordinates. This graph embedding network, also called Struc2vec (S2V), combines the environment state with the previously developed tour graph [19]. This encoder-decoder network maps each customer in a tour graph into a low-dimensional vector in sequence with attention mechanism [53].

To deal with the partially observable decision-making of the DU-VRP, an improved overall process of RNN is developed based on the graph embedding method in this study, are displayed in Fig. 6. The S2V and the RNNs are determined by the encoder network. However, due to conventional RNNs' vanishing gradients, RNNs are limited in their ability to learn long-term relationships [54]. Long Short Term Memory (LSTM) is a concept that is used in RNNs cells to solve long-term dependence difficulties and carry information across numerous time steps. The LSTM cells in the RNN ensure the customer information to handle the input context from the S2V network, which is computed as a p-dimensional feature embedding such as μ_i^t, for each node c_i at time step t in a graph G. The current nodes are aggregated recursively according to the graph's topology. After some iterations, a new p-dimensional feature embedding for each node will be produced by the S2V network. The S2V network initializes the embedding $\mu_i^0 = 0$ for all

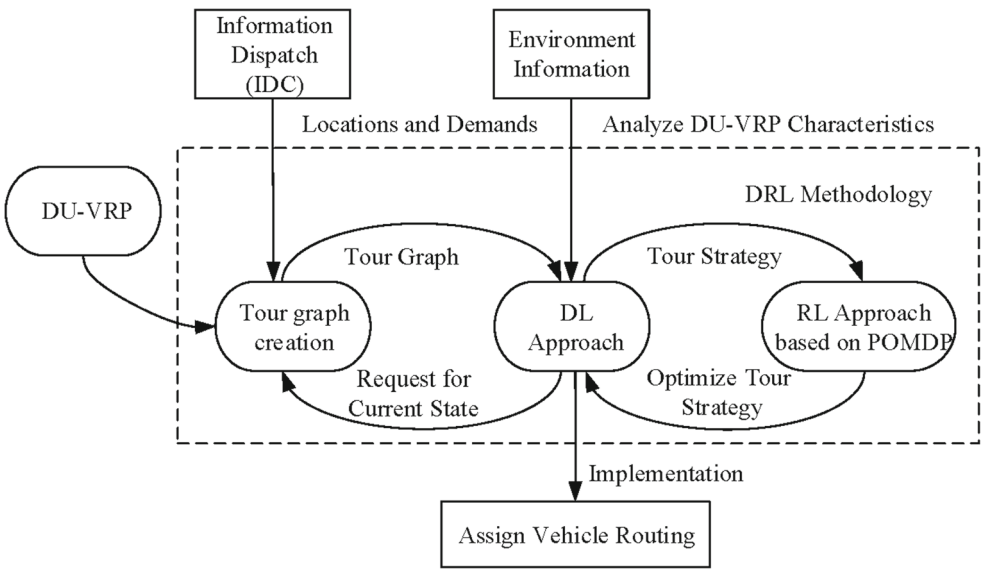


Fig. 4 The structure of the DRL methodology for the DU-VRP

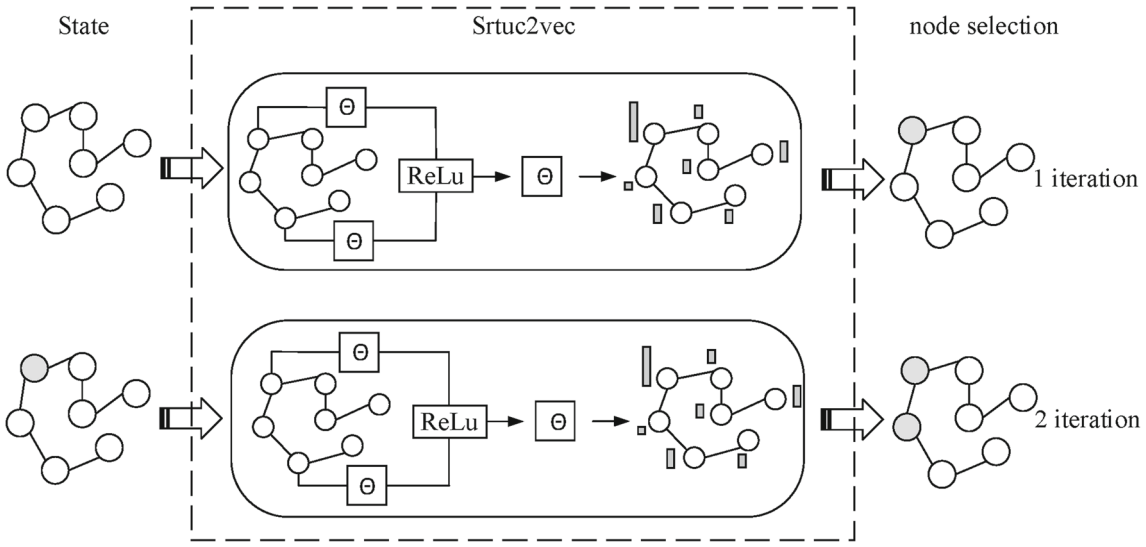


Fig. 5 The process of the input layer in the deep learning for the DU-VRP

elements of $\mathbb{C}$, considering both current graph information and long-range interactions between these nodes. All embeddings update synchronously at each iteration, as shown in Eq. (1):

$$\mu_i^{t+1} \leftarrow f(c_i, \{\mu_j^t\}_{j\in\mathcal{N}(i)}, \{\tilde{m}_{ij}^t\}_{j\in\mathcal{N}(i)}, \tilde{d}_i^t; \Theta) \tag{1}$$

where $\mathcal{N}(i)$ expresses the set of neighbors of node c_i in G, $\tilde{m}_{ij}^t$ represents the observed distance between customer i and customer j at time step t, $\tilde{d}_i^t$ represents the observed demand of customer i at time step t, and $f()$ is a nonlinear function parameterized by Θ. The features of the nodes c_i are propagated to other nodes through a nonlinear function.

After the Tth iteration, the embedding of each node will contain information regarding its neighbors on the graph, including node features and propagation functions. The nonlinear function $f()$, which is used to update the embedding μ_i is defined as shown in Eq. (2):

$$\mu_i^{t+1} \leftarrow ReLU(\theta_1 c_i + \theta_2 \sum_{j\in\mathcal{N}(i)} \mu_j^t + \theta_3 \sum_{j\in\mathcal{N}(i)} ReLU(\theta_4 \tilde{d}_i^t) + \theta_5 \sum_{j\in\mathcal{N}(i)} ReLU(\theta_6 \tilde{m}_{ij}^t)) \tag{2}$$

where $\theta_1 \in R^p$; $\theta_2, \theta_3, \theta_5 \in R^{p\times p}$;and $\theta_4, \theta_6 \in R^p$ represent the dimensional model vectors, which are represented by the parameter Θ. The ReLU function ($ReLU(z) = \max\{0, z\}$) is applied to accelerate the convergence of the embedding node. Then the embeddings for each node are

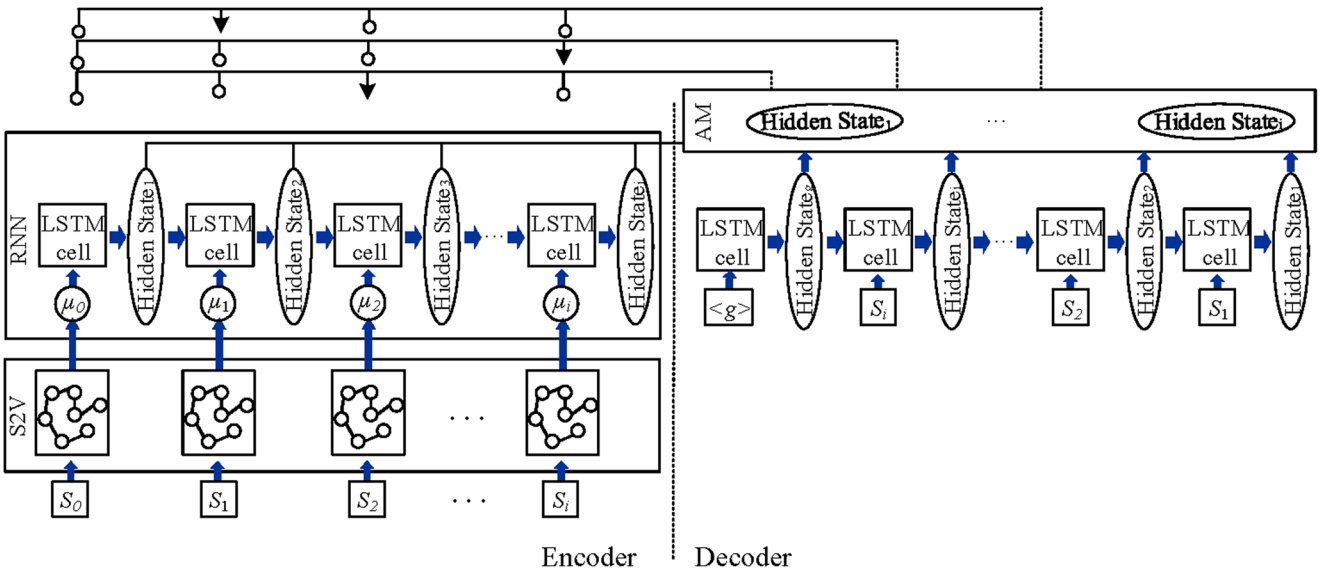


Fig. 6 The process of an improved neural network for the DU-VRP

input into an RNN, which consists of LSTM that uses the input data to create a sequence of p-dimensional latent memory states $C_i^e \in \mathbb{R}^p$ in the encoder network. Equations (3) to (8) are the internal expressions of the LSTM cells:

$$f_i = \sigma(W_f \cdot \left[h_{i-1}, \mu_i^T\right] + b_f) \tag{3}$$

$$J_i = \sigma(W_j \cdot \left[h_{i-1}, \mu_i^T\right] + b_j) \tag{4}$$

$$\widetilde{C}_i = tanh(W_C \cdot \left[h_{i-1}, \mu_i^T\right] + b_C) \tag{5}$$

$$C_i^e = f_i * C_{i-1}^e + J_i * \widetilde{C}_i \tag{6}$$

$$o_i = \sigma(W_O \left[h_{i-1}, \mu_i^T\right] + b_O) \tag{7}$$

$$h_i = o_i * tanh(C_i^e) \tag{8}$$

where $\sigma(z) = 1/(1 + e^{-z})$ is a sigmoid function, and tanh is an activation function that converts the input to a real-value between -1 and 1. The symbol $*$ indicates the multiplication of the corresponding elements of the operation; W_f, W_j, W_C, W_O denote the weight parameters; and b_f, b_j, b_C, b_O denote the bias parameters. A set of state vectors $S = (S_1, S_2, \ldots, S_t)$ with hidden states $H = (h_1, h_2, \ldots, h_t)$ is produced when the embedding node passes through the encoder.

Next, the LSTMs cells are adopted to develop the decoder, which is decoding the encoder context and outputting the predicted sequence to the nodes in tours. The decoder constructs a series of p-dimensional latent memory by Eq. (2). Following that, the decoder is used to decode the encoders' input and output the tour in an iterative fashion. In each iteration, the decoder applies a series of p-dimensional latent states of the current partial constructed customer $y\,(< i)$ to predict the distribution of the next node.

To predict a distribution among the customers from the encoding context, we improve an attention mechanism (AM) method proposed by Nazari et al. [26]. The encoder part only contains the coordinates and demands information at the current iteration and the decoder part only remembers static information from the past in the Nazari et al. [26]. In the output layer of the DL approach, the AM is applied to compute the alignment vector in the decoder network. We design an attention mechanism with a pooling layer to modify the input of the decoder network. All the hidden states C_t^i consist of the static (the states information) and dynamic (the observation states information) element $C_i^e \in \mathbb{R}$ from each iteration in the encoder. C_t^i is important as it is a combination of the outputs from the encoder and decoder, containing the previous information about customers' demands. Before feeding the hidden state into the decoder, we first apply a pooling layer on one of the dimensions of the hidden state and make sure it has the same shape as the decoder input. The encoder and decoder hidden state $C_i^d \in \mathbb{R}$ are combined to determine the alignment vector as follow by Eqs. (9)-(12).

$$C_t' = pooling\left(C_t^i\right) \tag{9}$$

$$a_t^i = v_a^T * tanh\left(W^e C_t' + W^d C_i^d\right) \tag{10}$$

$$a_t\left(\left\{C_i^e\right\}_{i=1}^n, C_i^d, W^e, W^d, v\right) = softmax(a_t^i) \tag{11}$$

$$c_t = \sum_{i=1}^{n} C_t' a_t^i \tag{12}$$

where $v_a \in R^p$ represents an attention vector and W^e, $W^d \in R^{p \times p}$ denote attention matrices. The function softmax is called an activation function that allocates the probability of visiting the next customer $y\,(i)$ according to Eq. (13):

$$p\left(y\,(i) \mid y\,(< i)\,, S\right) = softmax\left(\widetilde{u_i}\right) \tag{13}$$

where $\widetilde{u_i} = v^T tanh\left(W_c\left[\left\{C_i^e\right\}_{i=1}^n, c_t\right]\right)$, $v \in R^p$ represents an attention vector. This probability distribution indicates that the decoder network is attending to the customer in the given state S in the form of a probability. Consequently, we use the rule of the probability chain to factorize the probability of a tour, as obtained by Eq. (14):

$$p\left(Y \mid S\right) = \prod_{i=1}^{n} p\left(y\,(i) \mid y\,(< i)\,, S\right) \tag{14}$$

3.2.3 An RL approach

We present an RL approach to train the model parameters of the proposed DL approach for DU-VRP. The RL approach, which is different from the supervised learning approach, uses a reward mechanism for the adaptive agents to improve their performance, instead of relying on optimal solutions to the DU-VRP. In this RL approach, as shown in Fig. 7, the agents can interact with the environmental changes controlled by a processor that consists of the graph-learning, sequence-learning, and policy-learning networks. The processor can observe the states from a dynamic DU-VRP environment. Subsequent to the observation, the updated state is the input to the graph-learning network. Then, the sequence-learning network reorders the updated state and delivers the outcome to the policy-learning network, which produces the optimal action to the environment.

To deal with the partially observable state of the DU-VRP, we develop a novel RL approach based on the POMDP model. The asynchronous advantage actor-critic (called "A3C") algorithm uses asynchronous gradient descent for optimization of deep neural network controllers based on actor-critic methods, introduced by Mnih [32], where an 'actor' refers to the learned policy, and 'critic' refers to add

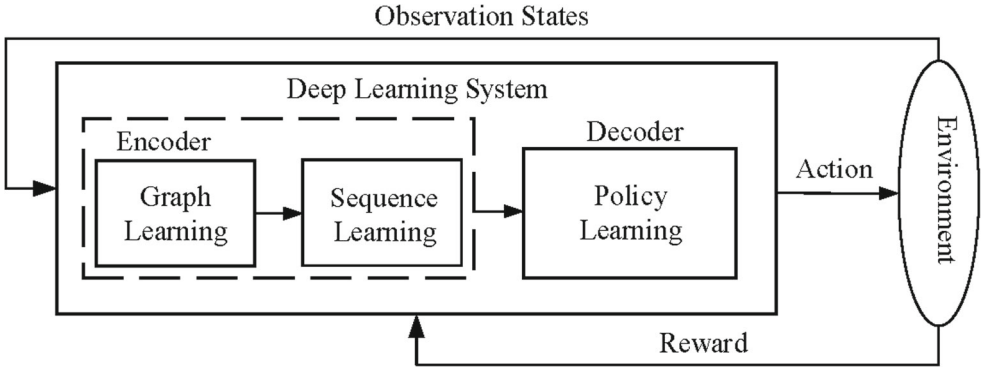


Fig. 7 Interaction agent-environment in the reinforcement learning for the DU-VRP

a value function to the REINFORCE policy gradient algorithm [13]. The actor develops a sample route throughout the training procedure. The value function, on the other hand, adjusts the intended rewards and improves itself using the created routes before revising the policy. The value function is then used to update the policy's parameters in the area of improving performance. Because the critic is modelled using a bootstrap approach, the variance is reduced, making the learning more reliable than vanilla policy gradient methods. The A3C algorithm allows many actor-critic agents to train in parallel. Each actor-critic has its own set of parameters and interacts with the environment in a unique way. A global network receives updates from each agent and combines them asynchronously to create a global policy. In this way, parallel actor-learners accumulate updates for improving training stability. A model-free policy-based RL approach by integrating the A3C algorithm is proposed to train the model parameters for the DU-VRP, as displayed in Fig. 8.

There are two sets of parameters in the A3C algorithm. In the DL method, we employ the symbol Γ to denote the collection of model parameters, including the graph embedding θ; LSTM parameters W, b; and attention parameters v, W. The performance measure is defined based on the policy gradient method as follow:

$$J(\Gamma) = \mathbb{E}_{\pi\sim\Gamma(.|o)} V_{\pi}(o) \tag{15}$$

The 'critic' is provided to estimate the action value function associated with parameters φ, which can train the model parameters Γ as 'actor'. The A3C algorithm follows an approximate policy gradient

$$\nabla_{\Gamma} J(\Gamma) = \mathbb{E}_{\pi\sim\Gamma(.|o)} \left[\nabla_{\Gamma} log\pi_{\Gamma}(Y \mid O)\, Q_{\varphi}(O, y)\right] \tag{16}$$

Then, we introduce a baseline function $B(O)$, which can reduce variance without changing expectations for the RL approach [53]. Typically, the baseline function is the state value function $B(O) = V_{\pi}(O, \varphi)$. Thus,

$$\nabla_{\Gamma} J(\Gamma) = \mathbb{E}_{\pi\sim\Gamma(.|O)} \left[\nabla_{\Gamma} log\pi_{\Gamma}(Y \mid O)\left(Q_{\varphi}(O, y) - V_{\pi}(O, \varphi)\right)\right] \tag{17}$$

The proposed A3C algorithm's pseudo-code is described below, where Γ^{o}, φ^{o} are the weights of a global network

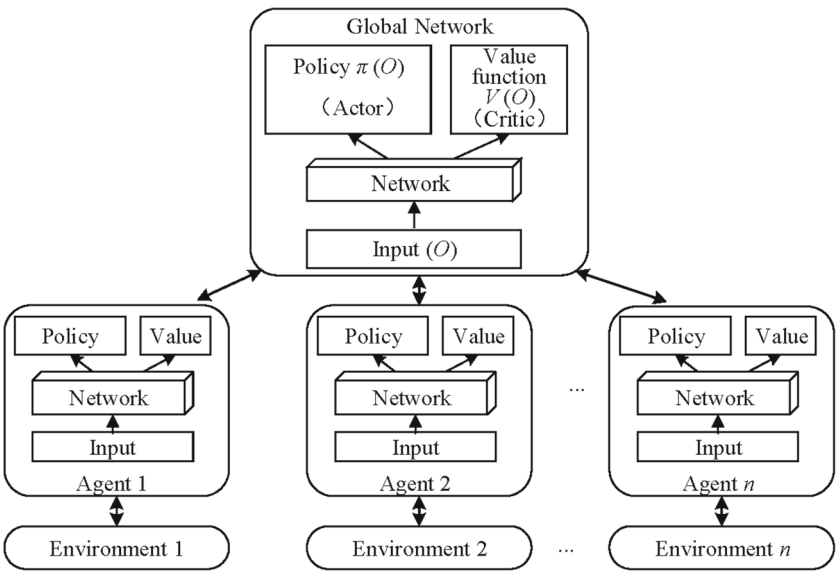


Fig. 8 The framework of the asynchronous advantage actor-critic algorithm

with weights assigned to the actor and critic, respectively. Besides, the agents runn in parallel threads with their own network parameters, are denoted as Γ^n, φ^n for agent n.

Algorithm 1 Asynchronous Advantage Actor-Critic Algorithm.

Input: a differentiable global network policy parameterization $\pi(Y \mid O, \Gamma^0)$; N thread-specific networks policy parameterizations $\pi(Y \mid O, \Gamma^n)$ for each thread n

Input: a differentiable global network state-value parameterization $V(O, \varphi^o)$; N thread-specific networks policy parameterizations $V(O, \varphi^n)$ for each thread n

Parameters: upper bound of global shared step counter T_{max}; step sizes $\alpha^\Gamma > 0, \alpha^\varphi > 0$; and thread step counter t_{local} in each thread

Initialize global shared step counter $T \leftarrow 0$
while $T <= T_{max}$ **do**
  for each thread $n \leftarrow 1$ to N **do**
    Initialize thread step counter $t \leftarrow 1$
    Reset gradients: $\Gamma^n \leftarrow \Gamma^0, \varphi^n \leftarrow \varphi^0$
    $t_{start} \leftarrow t$
    Initialize o_t
    while o_t is non-terminal or $t - t_{start} == t_{max}$ **do**
      Take action y_t according to policy $\pi(y_t \mid o_t, \Gamma^n)$
      Observe reward R_t and new state o_{t+1}
      $t \leftarrow t + 1$
      $T \leftarrow T + 1$
    end while
    $\mathcal{R} = V(o_t, \varphi^n)$
    for $i \leftarrow t - 1$ to t_{start} **do**
      $\mathcal{R} = \mathcal{R} + R_i$
      $d\Gamma^0 \leftarrow d\Gamma^0 + \nabla_{\Gamma^n} log\pi_{\Gamma^n}(y_i \mid o_i)(\mathcal{R} - V(O_i, \varphi^n))$
      $d\varphi^0 \leftarrow d\varphi^0 + \frac{\partial(\mathcal{R} - V(O_i, \varphi^n))^2}{\partial \varphi^n}$
    end for
    Perform asynchronous update $\Gamma^0, \varphi^0 : \Gamma^0 \leftarrow \Gamma^0 - \alpha^\Gamma d\Gamma^0; \varphi^0 \leftarrow \varphi^0 - \alpha^\varphi d\varphi^0$
  end for
end while

4 Experiments and results

This section conducts three groups of experiments to assess the proposed DRL model's performance. The initial set of experiments is designed to train the DRL model. The performance of the DRL model is then demonstrated on the basic VRP dataset in the second set of experiments. Finally, the third groups of experiments is implemented in real-world scenarios to ensure that the DRL method is successful. All programs are coded in PyCharm with Python 3.7 and Pytorch 1.8, and the results reported below are obtained on a PC with an Intel Core i7 processor and 32GB of RAM.

4.1 Experimental parameters

Since the DU-VRP is a relatively new topic with no standard dataset, the initial set of experiments uses randomly generated cases. Meanwhile, customer locations are created at random on a $1 * 1$ map, with the depot as the first node and the rest as customer nodes. The customers' demand is a random variable parameter within [1, 9]. The vehicles have a capacity ranging from 20 to 60. Finally, vehicles of various capacities will arrive at the depot and transport items to all customers.The configuration of the parameters in the initial set of experiments is shown in Table 2.

The initial set of experiments focuses on creating an optimal strategy to control the DRL method. During the procedure, parallel actor-learners in the proposed algorithm are introduced to speed up the proposed technique, and randomly generated issues are employed as training cases. Many parameters in the proposed technique must be adjusted in order to reach the optimal outcome. These parameters are chosen according to the training process. Table 3 shows the values of parameters.

We save and test the RL approach after the training process to see how it performs. The experiment instance is presented in Table 4 during the test phase. Table 5 shows the average tour lengths of our proposed DRL.

The proposed DL approach can handle the DU-VRP efficiently by applying RL for generating better policies, as shown in Table 5.

4.2 Performance on benchmarking datasets

In this second set of the experiment, we adopt the RNNs with 128 hidden units in the encoder network, which corresponds to the number of nodes in the graph embedding.

Table 2 The experiment configuration of DU-VRP instances

Entity	Property	Value
Environment	Shape	Square
	Space	(0,0) to (1,1)
	Deport	(0.5,0.5)
Customers	Number of customers	10 to 100
	Customer location	(0,0) to (1,1)
	Customer demand quantity	1 to 9
Vehicles	Number of vehicles	2 to 6
	Vehicle capacity	20 to 60

Table 3 Parameters configuration for the proposed DRL

Entity	Property	Value
RL strategy	Actor learning rate	0.001
	Actor learning rate	0.001
	Batch size for updating	128
	Train size for updating	100
	Number of problems in the test set	1000
	Number of actor-learners	4
DL strategy	Dimension of input embedding	128
	Max episode	60
	Dimension of hidden layers in Encoder	128
	Dimension of hidden layers in Decoder	128
	Number of process block iteration	3
	Episode decision moment number	2*customer number
	Number of LSTM layers in the encoder	2
	Number of LSTM layers in the decoder	1

The inputs by small batch B=128 are generated using the benchmark data set during the training. These small batches directly determine the weights of our model in optimization. Each static (the states information) element consisting of the customer coordinate is embedded into a vector with 128 hidden units sharing among the inputs. The dynamic (the observation states information) element is then embedded in the same way. The dynamic elements including the customers' demand, the remaining load of vehicles are mapped to a vector of size 128. For the decoder network, we employ a layer of 128 hidden units in the LSTM. In the actor and critical networks, we initialize two types of settings, either by using greedy search for decoding, or by applying the best result from a batch of sampled results as the softmax results. We initialise two types of settings in the actor and critic networks: greedy search for decoding and using the optimal performance from a set of observed outputs as the softmax results.

To evaluate the proposed A3C performance, the different numbers of customers with the same vehicle are introduced. To show the overall performance of DRL, an evaluation index called the gap, which is calculated between the DRL and the OPT by (18), where X means the DRL, T(X) denotes the average route time of X and OPT denotes the best solution.

$$GAP = \frac{T(X) - T(OPT)}{T(OPT)} \tag{18}$$

This sort of difficulty is rather common, and a typical benchmark can be from the website of the NEO (cf.https://neo.lcc.uma.es/vrp/vrp-instances/). We apply the two benchmark data sets (Set-A and Set-B) to verify the proposed DRL methodology. The proposed A3C algorithm is modified in the second set of experiments to fit the DU-VRP situation. Besides, we set the dynamic scenarios to verify the proposed algorithm. Furthermore, the benchmark's best solution is utilised to illustrate the algorithm's overall level. All instances are utilised to evaluate the performance of all algorithms during the experiment. The number of entities (vehicles) is limited for each instance to ensure that all algorithms use the same computer capacity and can meet demand changes. First, we apply the data of Set-A to verify the performance of DRL. Besides, the OPT is obtained from the website, the table column 'KX' represents there are X vehicles, and the instance named 'A-nY' means the data with Y customers. The table columns refer respectively to: the Entity (Vehicles), the number of instances in the entity (No.), the instances name(Instances), the GAP(%), the value of the total traveling distance (Distance) obtained with OPT and the DRL, and the computational time in seconds (Time). Table 6 shows the performance of the algorithms while solving the data of Set-A.

The partial observations are obtained and shown in Table 6. It is observed that DRL can achieve satisfactory results in a very short time. For the instances with number

Table 4 Testing process of the experimental instances

Name	No. Customers	No. Vehicles	Capacities
VRP10	10	2	20
VRP20	20	3	30
VRP50	50	4	40
VRP80	80	5	50
VRP100	100	6	60

Table 5 Average tour length of proposed DRL

Baseline	VRP10		VRP20		VRP50		VRP80		VRP100	
	mean	std	mean	std	mean	std	mean	std	mean	std
DRL	5.058	1.015	6.980	1.368	8.992	1.562	15.581	1.821	18.025	1.926

of customers 32, 34 in K5 the performance of DRL methodology is close to OPT. In particular, the proposed A3C RL approach, in particular, outperforms the benchmark, and improves or reaches the adopting a strategic in all cases. The gap between the A3C RL approach and the OPT from 1.58% to 9.75% while the average gap is 5.79% in these cases. To summarise, the proposed A3C RL approach can produce an satisfactory solution in a short amount of time and its usefulness has been demonstrated.

Then, using the additional data from Set-B, we evaluate the A3C RL approach performance with different amounts of demands. The performance of the algorithms in solving the data of Set-B instances with the various scales (instance types and number of customers) are analyzed in Table 7, which indicates that the 'KX' represents the X vehicles and the instance named 'B-nY' means the instance with Y customers.

From Table 7, it is observed that the GAP for these VRP instances is controlled by the 13%. All instances are controlled by the 10% except the number of customer 39 in K5. For the instances with the number of customers 31, 34 and 35 in K5 the performance of A3C RL approach is close to OPT. Specifically, in the case of a certain the number of vehicles, the calculation time increases with the increase of customer demand, but the GAP between DRL and OPT is very small. Then, as the number of vehicles increases, so

Table 6 Performance of the algorithms while solving the benchmarking instances

Entity (Vehicle)	No.	Instance	GAP (%)	OPT	DRL	Time(s)
K5	1	A-n32	1.58	784	796.4	0.297
	2	A-n33	4.79	661	692.7	0.311
	3	A-n34	2.78	778	799.7	0.313
	4	A-n36	4.08	799	831.6	0.346
	5	A-n37	8.75	669	727.6	0.351
	6	A-n38	8.79	730	794.2	0.374
	7	A-n39	6.27	822	873.6	0.401
K6	1	A-n44	7.68	937	1009.0	0.452
	2	A-n45	6.99	944	1010.0	0.495
K7	1	A-n46	6.71	914	975.4	0.485
	2	A-n48	6.31	1073	1140.8	0.546
	3	A-n53	8.78	1010	1098.7	0.599
	4	A-n54	8.48	1167	1266	0.624
K8	1	A-n62	8.29	1288	1394.8	0.788
K9	1	A-n55	6.73	1073	1145.3	0.677
	2	A-n60	8.78	1354	1472.9	0.791
	3	A-n61	9.42	1034	1131.5	0.793
	4	A-n63	8.31	1616	1750.4	0.875
	5	A-n64	5.50	1401	1478.1	0.851
	6	A-n65	8.55	1174	1274.4	0.921
	7	A-n69	6.16	1159	1230.4	0.976
K10	1	A-n80	9.75	1763	1935.0	1.22

Table 7 Performance of the algorithms while solving the benchmarking instances

Entity (Vehicle)	No.	Instance	GAP (%)	OPT	DRL	Time(s)
K5	1	B-n31	3.73	672	697.1	0.265
	2	B-n34	3.32	788	814.2	0.319
	3	B-n35	3.32	788	814.2	0.319
	4	B-n39	12.7	549	619.2	0.421
	5	B-n45	5.07	751	789.1	0.484
K6	1	B-n38	7.03	805	861.6	0.359
	2	B-n41	7.65	829	892.5	0.426
	3	B-n43	7.66	742	798.9	0.454
K7	1	B-n44	5.51	909	959.1	0.460
	2	B-n50	5.23	741	779.8	0.571
	3	B-n51	5.41	1032	1087.8	0.594
	4	B-n52	9.12	747	815.2	0.570
	5	B-n56	8.48	1167	1266	0.624
	6	B-n57	7.55	1153	1240.1	0.709
K9	1	B-n64	8.75	861	936.4	0.817
	2	B-n66	5.44	1316	1387.7	0.89
	3	B-n68	7.14	1272	1362.9	0.944
K10	1	B-n63	7.68	1496	1610.9	0.859
	2	B-n67	7.68	1496	1610.9	0.859
	3	B-n78	8.08	1221	1319.7	1.18

does the demand of customers, and the GAP between DRL and OPT is within 10%. After that, the GAP value is always within 10%, indicating that DRL can meet the solution quality under the change of demand. As a consequence, the proposed A3C RL approach can produce outstanding outcomes under different customers' demands. Finally, the outcome is displayed in Table 7, and it appears to be satisfactory and supports the A3C RL approach's efficacy. To summarise, the proposed A3C RL approach can produce an satisfactory solution in a short amount of time and its usefulness has been demonstrated.

4.3 Performance in real-world scenarios

The third group of experiments is shown in a real-world transportation network using data collected from Cologne, Germany (cf. https://sumo.dlr.de/docs/Data/Scenarios/TAPASCologne.html). We adopt the same RNNs with 128 hidden units in the encoder network, which is consistent with the number of nodes in the graph embedding. The inputs by small-batch B=128 are generated using the Cologne data set during the training. These small batches directly determine the weights of our model in optimization.

4.3.1 Experimental settings

All of the experiments in the third group use the data of Cologne, Germany. The real-world data are obtained from the OpenStreetMap file [39]. The latitude and longitude of each node in this real-world map are applied to construct the actual roadmap. A Python package is used to download administrative boundary shapes and street networks from Osmnx [55]. The sampling size is divided into five-minute parts from 8:00 am to 10:00 pm and all demands have been satisfied. Averaging all speeds that can be matched to the route in the relevant time segment using GPS records yields the real-time traffic speed of any arbitrary road. In the proposed DRL experiment, we adopt the RNNs with 128 hidden units in the encoder network, which is consistent with the number of nodes in the graph embedding. The inputs by small-batch B=128 are generated using the small transport network during the training. These small batches directly determine the weights of our model in optimization. We run 3,000 training steps to train the DRL model on the Cologne dataset, and set the buffer size to 5,000 experiences. We apply the Adam optimizer [56] with the learning rate of 1e-3 to train encoder and decoder network. Over

Table 8 Performance of the algorithms while solving real-world instances

Strategy	No. customers	Distance (km)	Time (s)
DRL-greedy	198	4987.56	2.89
DRL-BS	200	5185.55	3.98
DRL-SS-100	200	4618.19	0.01
DRL-SS-300	200	4520.23	4.9
DRL-SS-500	200	3966.52	10.8
DRL-SS-1280	200	3666.32	38.1
DRL-SS-12800	200	3541.23	237.5

the training stages, the ϵ-greedy [28] policy with a decay rate of 0.9995 is utilised. There are N $\times$ N vertex generating procedures in each episode. We suppose that the initial node is the depot. The test case obtained by Osmnx is used to construct a tour graph for parameters adjustment in graph-embedding-network by A3C Algorithm. The training process is repeated 10000 episodes for approximately 45 hours. At the start of each episode, the graph is randomly generated as input with corresponding demands.

4.3.2 Solution quality

We test the experiments in different tour sampling situations with various customers' demands in the same transportation network to show the quality of the A3C RL approach. The customers' locations are randomly selected from nodes in the Cologne map. We design three types of sampling strategies to generate test cases: (i) greedy, which chooses the node with the maximum probability as another location; (ii) beam search (BS), which takes account of the most likely paths and then chooses the node with the shortest distance length; and (iii) sample search (SS), which selects M candidate tours based on the possible outcomes established by the probability policy $p\left(y\left(i\right) \mid y\left(<i\right), S\right)$. We test the experiments with candidates 100, 300, 500, 1280, or 12800, respectively, using the sample search strategy for the DU-VRP. The performance of the algorithms while solving Cologne data set with the various scales (e.g., strategy choices and instance sizes) are analyzed in Table 8, which the Stategy named 'DRL-X-Y' means the DRL with X strategies and with Y candidates. In summary, the proposed DRL methodology is applicable to solve the DUVRP in real time.

5 Conclusion

In this paper, we introduce and define a more generalized problem called the dynamic and uncertain vehicle routing problem (DU-VRP), which covers the classical vehicle routing problem with dynamic environment and uncertain customers' demands. We design a partially observed Markov decision process (POMDP) to reflect the partial observation states in the DU-VRP environment. The proposed DRL methodology combines the merits of a deep learning approach and a reinforcement learning approach to learn and optimize the routing decisions in a dynamic and uncertain environment. Computational results show that the proposed DRL methodology can tackle the curse of dimensionality because it can handle the partial observation of the state space with an improved attention mechanism method.

Regarding future research directions, other machine learning approaches for training deep neural networks will be investigated in depth. The proposed DRL methodology would be extended to tackle more real-world applicable VRPs with incomplete traffic information [57, 58]; to solve other routing-production-related combinatorial optimization problems such as job shop scheduling with the parallel use of resources and sequence-dependent setup times [59, 60]; and to deal with industry-oriented mining optimisation problems [61–63].

Acknowledgments This work is supported by the National Natural Science Foundation of China under Grant No. 71871064.

Declarations

Conflict of Interests The authors declared that they have no conflicts of interest to this work.

References

1. Steever Z, Karwan M, Murray C (2019) Dynamic courier routing for a food delivery service. Comput Oper Res 107:173–188. https://doi.org/10.1016/j.cor.2019.03.008
2. Drent C, Keizer MO, Houtum GJ van (2020) Dynamic dispatching and repositioning policies for fast-response service networks. Eur J Oper Res 285:583–598. https://doi.org/10.1016/j.ejor.2020.02.014
3. Hong J, Lee M, Cheong T, Lee HC (2019) Routing for an on-demand logistics service. Transp Res Part C: Emerg Technol 103:328–351. https://doi.org/10.1016/j.trc.2018.12.010
4. Zhang Z, Sun Y, Xie H, Teng Y, Wang J (2018) Gmma: Gpu-based multiobjective memetic algorithms for vehicle routing problem with route balancing. Applied Intelligence 49:63–78. https://doi.org/10.1007/s10489-018-1210-6
5. Vidal T, Laporte G, Matl P (2020) A concise guide to existing and emerging vehicle routing problem variants. Eur J Oper Res 286:401–416. https://doi.org/10.1016/j.ejor.2019.10.010
6. Nasri M, Metrane A, Hafidi I, Jamali A (2020) A robust approach for solving a vehicle routing problem with time windows with uncertain service and travel times. Int J Ind Eng Comput 11:1–16. https://doi.org/10.5267/j.ijiec.2019.7.002
7. Mazyavkina N, Sviridov S, Ivanov S, Burnaev E. (2021) Reinforcement learning for combinatorial optimization: A survey. Comput Oper Res 134:105400. https://doi.org/10.1016/j.cor.2021.105400

8. Karimi-Mamaghan M, Mohammadi M, Meyer P, Karimi-Mamaghan AM, Talbi E-G (2022) Machine learning at the service of meta-heuristics for solving combinatorial optimization problems: a state-of-the-art. Eur J Oper Res 296:393–422. https://doi.org/10.1016/j.ejor.2021.04.032
9. Wang Q, Tang C (2021) Deep reinforcement learning for transportation network combinatorial optimization: a survey. Knowl-Based Syst 233:107526. https://doi.org/10.1016/j.knosys.2021.107526
10. Parvez Farazi N, Zou B, Ahamed T, Barua L (2021) Deep reinforcement learning in transportation research: a review. Transp Res Interdiscip Perspect 11:100425. https://doi.org/10.1016/j.trip.2021.100425
11. Ulmer MW, Thomas BW (2020) Meso-parametric value function approximation for dynamic customer acceptances in delivery routing. Eur J Oper Res 285:183–195. https://doi.org/10.1016/j.ejor.2019.04.029
12. Ning C, You F (2019) Optimization under uncertainty in the era of big data and deep learning: when machine learning meets mathematical programming. Comput Chem Eng 125:434–448. https://doi.org/10.1016/j.compchemeng.2019.03.034
13. Sutton RS, Barto AG (2018) Reinforcement learning part II an introduction, 2nd edition. MIT press, Cambridge
14. Schrittwieser J, Antonoglou I, Hubert T, Simonyan K, Sifre L, Schmitt S, Guez A, Lockhart E, Hassabis D, Graepel T, Lillicrap T, Silver D (2020) Mastering atari, go, chess and shogi by planning with a learned model. Nature 588:604–609. https://doi.org/10.1038/s41586-020-03051-4
15. Hubmann C, Schulz J, Becker M, Althoff D, Stiller C (2018) Automated driving in uncertain environments: planning with interaction and uncertain maneuver prediction. IEEE Trans Intell Veh 3:5–17. https://doi.org/10.1109/TIV.2017.2788208
16. Pouya P, Madni AM (2021) Expandable-partially observable Markov decision-process framework for modeling and analysis of autonomous vehicle behavior. IEEE Syst J 15:3714–3725. https://doi.org/10.1109/JSYST.2020.3010473
17. Bengio Y, Lodi A, Prouvost A (2021) Machine learning for combinatorial optimization: a methodological tour d'horizon. Eur J Oper Res 290:405–421. https://doi.org/10.1016/j.ejor.2020.07.063
18. Bui Khac Hoai N, Cho J, Yi H (2021) Spatial-temporal graph neural network for traffic forecasting: an overview and open research issues. Applied Intelligence 52:2763–2774. https://doi.org/10.1007/s10489-021-02587-w
19. Vesselinova N, Steinert R, Perez-Ramirez DF, Boman M (2020) Learning combinatorial optimization on graphs: a survey with applications to networking. IEEE Access 8:120388–120416. https://doi.org/10.1109/ACCESS.2020.3004964
20. Sun P, Hu Y, Lan J, Tian L, Chen M (2019) Tide: time-relevant deep reinforcement learning for routing optimization. Futur Gener Comput Syst 99:401–409. https://doi.org/10.1016/j.future.2019.04.014
21. Huynh TT, Lin CM, Lee K, The Vu M, Nguyen N, Chao F (2021) Intelligent wavelet fuzzy brain emotional controller using dual function-link network for uncertain nonlinear control systems. Applied Intelligence 52:2720–2744. https://doi.org/10.1007/s10489-021-02482-4
22. Xu R, Li M, Yang Z, Yang L, Qiao K, Shang Z (2021) Dynamic feature selection algorithm based on q-learning mechanism. Appl Intell 51:1–12. https://doi.org/10.1007/s10489-021-02257-x
23. Wang Q (2021) Varl: a variational autoencoder-based reinforcement learning framework for vehicle routing problems. Appl Intell. https://doi.org/10.1007/s10489-021-02920-3
24. Chen L, Cui J, Tang X, Qian Y, Li Y, Zhang Y (2021) Rlpath: a knowledge graph link prediction method using reinforcement learning based attentive relation path searching and representation learning. Applied Intelligence 52:4715–4726. https://doi.org/10.1007/s10489-021-02672-0
25. Zhang K, He F, Zhang Z, Lin X, Li M (2020) Multi-vehicle routing problems with soft time windows: a multi-agent reinforcement learning approach. Transportation Research Part C: Emerging Technologies 121:102861. https://doi.org/10.1016/j.trc.2020.102861
26. Nazari M, Oroojlooy A, Snyder LV, Takáč M (2018) Reinforcement learning for solving the vehicle routing problem. In: Proceedings of the 32nd international conference on neural information processing systems (NeurIPS 2018). Montréal, pp 9861–9871. https://dl.acm.org/doi/10.5555/3327546.3327651
27. Zhao J, Mao M, Zhao X, Zou J (2021) A hybrid of deep reinforcement learning and local search for the vehicle routing problems. IEEE Trans Intell Transp Syst 22:7208–7218. https://doi.org/10.1109/TITS.2020.3003163
28. Dai H, Khalil EB, Zhang Y, Dilkina B, Song L (2017) Learning combinatorial optimization algorithms over graphs. Adv Neural Inf Process Syst. Red Hook, pp 6349–6359. https://dl.acm.org/doi/10.5555/3295222.3295382
29. Csy A, Ahfc B, Ksc A (2020) An actor-critic deep reinforcement learning approach for metro train scheduling with rolling stock circulation under stochastic demand. Transp Res B Methodol 140:210–235. https://doi.org/10.1016/j.trb.2020.08.005
30. Kullman ND, Cousineau M, Goodson JC, Mendoza JE (2021) Dynamic ride-hailing with electric vehicles. Transp Sci 1–20. https://doi.org/10.1287/trsc.2021.1042
31. Ulmer MW, Goodson JC, Mattfeld DC, Thomas BW (2020) On modeling stochastic dynamic vehicle routing problems. EURO J Transp Logist 9:100008. https://doi.org/10.1016/j.ejtl.2020.100008
32. Mnih V, Badia AP, Mirza M, Graves A, Harley T, Lillicrap TP, Silver D, Kavukcuoglu K (2016) Asynchronous methods for deep reinforcement learning. In: Proceedings of the 33rd international conference on international conference on machine learning ICML 2016. New York, pp 1928–1937. https://dl.acm.org/doi/10.5555/3045390.3045594
33. Archetti C, Feillet D, Mor A, Speranza MG (2020) Dynamic traveling salesman problem with stochastic release dates. Eur J Oper Res 280:832–844. https://doi.org/10.1016/j.ejor.2019.07.062
34. Qiu H, Wang S, Yin Y, Wang D, Wang Y (2022) A deep reinforcement learning-based approach for the home delivery and installation routing problem. Int J Prod Econ 244:108362. https://doi.org/10.1016/j.ijpe.2021.108362
35. Yu JJQ, Yu W, Gu J (2019) Online vehicle routing with neural combinatorial optimization and deep reinforcement learning. IEEE Trans Intell Transp Syst 20:3806–3817. https://doi.org/10.1109/TITS.2019.2909109
36. Ahamed T, Zou B, Farazi NP, Tulabandhula T (2021) Deep reinforcement learning for crowdsourced urban delivery. Transp Res B Methodol 152:227–257. https://doi.org/10.1016/j.trb.2021.08.015
37. Silva ML, Souza SD, Souza MF, Bazzan A (2019) A reinforcement learning-based multi-agent framework applied for solving routing and scheduling problems. Exp Syst Applic 131:148–171. https://doi.org/10.1016/j.eswa.2019.04.056
38. Kang Y, Lee S, Chung BD (2019) Learning-based logistics planning and scheduling for crowdsourced parcel delivery. Comput Industr Eng 132:271–279. https://doi.org/10.1016/j.cie.2019.04.044
39. Zou G, Tang J, Yilmaz L, Kong X (2021) Online food ordering delivery strategies based on deep reinforcement learning. Appl Intell. https://doi.org/10.1007/s10489-021-02750-3
40. Wang Z, Qin Z, Tang X, Ye J, Zhu H (2018) Deep reinforcement learning with knowledge transfer for online rides order dispatching. In: Proceedings IEEE international conference data mining. ICDM, pp 617–626. https://doi.org/10.1109/ICDM.2018.00077
41. Liang E, Wen K, Lam WHK, Sumalee A, Zhong R (2021) An integrated reinforcement learning and centralized programming

approach for online taxi dispatching. IEEE Trans Neural Networks Learn Syst. https://doi.org/10.1109/TNNLS.2021.3060187
42. Turan B, Pedarsani R, Alizadeh M (2020) Dynamic pricing and fleet management for electric autonomous mobility on demand systems. Transp Res Part C: Emerg Technol 121:102829. https://doi.org/10.1016/j.trc.2020.102829
43. Chen X, Ulmer MW, Thomas BW (2022) Deep q-learning for same-day delivery with vehicles and drones. Eur J Oper Res 298: 939–952. https://doi.org/10.1016/j.ejor.2021.06.021
44. Liu Z, Li J, Wu K (2020) Context-aware taxi dispatching at city-scale using deep reinforcement learning. IEEE Trans Intell Transp Syst 23:1–14. https://doi.org/10.1109/TITS.2020.3030252
45. Liu X, Zhang D, Zhang T, Cui Y, Chen L, Liu S (2021) Novel best path selection approach based on hybrid improved a* algorithm and reinforcement learning. Applied Intelligence 51:9015–9029. https://doi.org/10.1007/s10489-021-02303-8
46. Tang X, Li M, Lin X, He F (2020) Online operations of automated electric taxi fleets: an advisor-student reinforcement learning framework. Transp Res Part C Emerg Technol 121:102844. https://doi.org/10.1016/j.trc.2020.102844
47. Koh S, Zhou B, Fang H, Yang P, Ji Z (2020) Real-time deep reinforcement learning based vehicle routing and navigation. Appl Soft Comput 96:106694. https://doi.org/10.1016/j.asoc.2020.106694
48. Mao C, Liu Y, Shen Z (2020) Dispatch of autonomous vehicles for taxi services: a deep reinforcement learning approach. Transportation Research Part C Emerging Technologies 115:102626. https://doi.org/10.1016/j.trc.2020.102626
49. Al-Abbasi AO, Ghosh A, Aggarwal V (2019) Deeppool: distributed model-free algorithm for ride-sharing using deep reinforcement learning. IEEE Trans Intell Transp Syst 20:4714–4727. https://doi.org/10.1109/TITS.2019.2931830
50. Basso R, Kulcsár B, Sanchez-Diaz I, Qu X (2022) Dynamic stochastic electric vehicle routing with safe reinforcement learning. Transp Res Part E: Logist Transp Rev 157:102496. https://doi.org/10.1016/j.tre.2021.102496
51. Guo P, Xiao K, Ye Z, Zhu W (2021) Route optimization via environment-aware deep network and reinforcement learning. ACM Transactions on Intelligent System and Technology 12:1–21. https://doi.org/10.1145/3461645
52. Hoel C-J, Driggs-Campbell K, Wolff K, Laine L, Kochenderfer MJ (2020) Combining planning and deep reinforcement learning in tactical decision making for autonomous driving. IEEE Trans Intell Veh 5:294–305. https://doi.org/10.1109/TIV.2019.2955905
53. Nguyen Q, Vien N, Dang V.-H., Chung T (2020) Asynchronous framework with reptile+ algorithm to meta learn partially observable Markov decision process. Appl Intell 50:4050–4062. https://doi.org/10.1007/s10489-020-01748-7
54. Wu X, Du Z, Guo Y, Fujita H (2019) Hierarchical attention based long short-term memory for chinese lyric generation. Appl Intell 49:44–52. https://doi.org/10.1007/s10489-018-1206-2
55. Boeing G (2017) Osmnx: new methods for acquiring, constructing, analyzing, and visualizing complex street networks. Comput Environ Urban Syst 65:126–139. https://doi.org/10.1016/j.compenvurbsys.2017.05.004
56. Melinte O, Vladareanu L (2020) Facial expressions recognition for human-robot interaction using deep convolutional neural networks with rectified adam optimizer. Sensors 20:2393. https://doi.org/10.3390/s20082393
57. Lesch V, König M, Kounev S et al (2022) Tackling the rich vehicle routing problem with nature-inspired algorithms. Appl Intell. https://doi.org/10.1007/s10489-021-03035-5
58. Zhang Q, Liu SQ, Masoud M (2020) A traffic congestion analysis by user equilibrium and system optimum with incomplete information. J Comb Optim. In press. https://doi.org/10.1007/s10878-020-00663-4
59. Liu CL, Chang CC, Tseng CJ (2020) Actor-critic deep reinforcement learning for solving job shop scheduling problems. IEEE Access 8:71752–71762. https://doi.org/10.1109/ACCESS.2020.2987820
60. Liu SQ, Kozan E (2016) Parallel-identical-machine job-shop scheduling with different stage-dependent buffering requirements. Comput Oper Res 74:31–41. https://doi.org/10.1016/j.cor.2016.04.023
61. Kozan E, Liu SQ (2017) An operational-level multi-stage mine production timetabling model for optimally synchronising drilling, blasting and excavating operations. Int J Mining, Reclam Environ 31:457–474. https://doi.org/10.1080/17480930.2016.1160818
62. Liu SQ, Kozan E (2019) Integration of mathematical models for ore mining industry. Int J Syst Sci Oper Logist 6:55–68. https://doi.org/10.1080/23302674.2017.1344330
63. Zeng L, Liu SQ, Kozan E et al (2021) A comprehensive interdisciplinary review of mine supply chain management. Resour Policy 74:102274. https://doi.org/10.1016/j.resourpol.2021.102274

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Weixu Pan is currently a third-year PhD student in Discipline of Management Science and Engineering at School of Economics and Management in Fuzhou university, China. He won the first-prize PhD Fellowship of Fuzhou University in 2019. He was a software engineer and has a solid foundation of expert knowledge and professional skills in machine learning.

Shi Qiang Liu is currently a professor with the title of Min-Jiang Scholar in the School of Economics and Management, Fuzhou University, China. He received the PhD degree in Operations Research from the School of Mathematical Sciences, Queensland University of Technology (QUT) in Australia. He was working in Australia for over 12 years as a Senior Research Scientist with CRC ORE and as a Research Fellow at QUT. He has published over 70 refereed papers, many in leading journals including Transportation Science (the foremost journal in Transportation), Decision Support Systems, International Journal of Production Economics, International Journal of Production Research, Expert Systems with Applications, Journal of the Operational Research Society, Computers & Operations Research, Computers & Industrial Engineering, Transportation Research Part E, IEEE Transactions on Automation Science and Engineering, Journal of Heuristics, Engineering Optimisation, Optimisation Letters, Advances in Engineering Software, etc. He has solid background in Operations Research and Management Science.